Variants of FFT and its applications

Yanqiao Chen
Southern University of Science and Technology
Department of Computer Science and Engineering
12412115@mail.sustech.edu.cn

April 11, 2026

Abstract

In this report, we present a systematic study of the Fast Fourier Transform (FFT) and its variants. We begin with a review of the fundamental principles underlying the FFT, followed by a detailed exposition of the Cooley-Tukey algorithm—an in-place iterative implementation that achieves $O(n \log n)$ time complexity. We further discuss the Number Theoretic Transform (NTT), a variant operating in finite fields that avoids floating-point arithmetic. Finally, we survey several applications of FFT-class algorithms across diverse domains, including quantum computing (Shor's algorithm), digital signal processing (FIR filters), image compression (DCT-based coding), and modern deep learning architectures (spectral convolution).

1 Foundations of the Fast Fourier Transform

1.1 Polynomial Multiplication and Convolution

The convolution operation is fundamental in signal processing and algebraic computations. Given two finite sequences $f = (f_0, f_1, \ldots, f_{n-1})$ and $g = (g_0, g_1, \ldots, g_{m-1})$, their discrete convolution is defined as:

$$(f * g)[k] = \sum_{i=0}^{k} f_i \cdot g_{k-i}, \quad 0 \leq k \leq n + m - 2$$

Direct computation of convolution requires $\Theta(nm)$ arithmetic operations. When $n = m$, this yields quadratic complexity $\Theta(n^2)$, which becomes prohibitive for large sequences.

The Convolution Theorem establishes a profound connection between convolution and the Fourier transform: for two sequences f and g,

$$\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g)$$

where $\mathcal{F}$ denotes the Discrete Fourier Transform (DFT) and $\cdot$ represents pointwise multiplication. This theorem enables the following efficient convolution algorithm:

1. **Padding:** Extend both sequences to length $N \geq n + m - 1$ (typically a power of 2) by appending zeros.
2. **Forward transform:** Compute $\hat{f} = \mathrm{FFT}(f)$ and $\hat{g} = \mathrm{FFT}(g)$.
3. **Pointwise multiplication:** Compute $\hat{h}[k] = \hat{f}[k] \cdot \hat{g}[k]$ for all k.

4. **Inverse transform:** Compute $h = \text{IFFT}(\hat{h})$ to obtain the convolution result.

This approach reduces the complexity from $O(n^2)$ to $O(N \log N)$, providing substantial efficiency gains for large inputs.

1.2 The Fast Fourier Transform

Let $P(x) = \sum_{j=0}^{n-1} a_j x^j$ be a polynomial of degree at most $n-1$, where $n = 2^m$ is a power of two. The DFT evaluates P at the n-th roots of unity $\omega_n^k = e^{-2\pi i k/n}$ for $k = 0, 1, \ldots, n-1$:

$$\hat{a}_k = P(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{kj}$$

The FFT exploits the recursive structure of the roots of unity through the *butterfly decomposition*. Separate $P(x)$ into even and odd indexed terms:

$$P_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2} x^{n/2-1}$$
$$P_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1} x^{n/2-1}$$

such that $P(x) = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2)$.

The crucial observation is the *halving lemma*: the squares of the n-th roots of unity yield the $(n/2)$-th roots of unity. Furthermore, the symmetry property states:

$$\omega_n^{k+n/2} = -\omega_n^k$$

Combining these properties yields the Cooley-Tukey butterfly formulas:

$$\hat{a}_k = P_{\text{even}}(\omega_{n/2}^k) + \omega_n^k \cdot P_{\text{odd}}(\omega_{n/2}^k) \tag{1}$$
$$\hat{a}_{k+n/2} = P_{\text{even}}(\omega_{n/2}^k) - \omega_n^k \cdot P_{\text{odd}}(\omega_{n/2}^k) \tag{2}$$

for $k = 0, 1, \ldots, n/2 - 1$. This decomposition reduces an n-point DFT to two $(n/2)$-point DFTs, yielding the recurrence $T(n) = 2T(n/2) + O(n)$ with solution $T(n) = O(n \log n)$.

1.3 Recursive FFT Algorithm

1.3.1 Algorithm Description

The recursive FFT follows the divide-and-conquer paradigm:

1. **Base case**: If $n = 1$, return the single coefficient $[a_0]$.
2. **Divide**: Split the polynomial into even and odd indexed coefficients:

$$P_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2} x^{n/2-1}$$

$$P_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1} x^{n/2-1}$$

3. **Conquer**: Recursively compute $\text{FFT}(P_{\text{even}}, n/2)$ and $\text{FFT}(P_{\text{odd}}, n/2)$.
4. **Combine**: Merge results using the butterfly operation with twiddle factors.

1.3.2 Step-by-Step Example: Recursive FFT for $n = 8$

Given coefficients $[a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7]$ and primitive root $\omega_8 = e^{-2\pi i/8}$.

Level 0: Initial Call ($n = 8$)
Split into:

- $P_{\text{even}} = [a_0, a_2, a_4, a_6]$ (even indices)
- $P_{\text{odd}} = [a_1, a_3, a_5, a_7]$ (odd indices)

Level 1: Recursive Calls ($n = 4$)
For $P_{\text{even}} = [a_0, a_2, a_4, a_6]$ with $\omega_4 = e^{-2\pi i/4}$:

- $P_{\text{even,even}} = [a_0, a_4]$ (indices 0, 4)
- $P_{\text{even,odd}} = [a_2, a_6]$ (indices 2, 6)

For $P_{\text{odd}} = [a_1, a_3, a_5, a_7]$ with $\omega_4 = e^{-2\pi i/4}$:

- $P_{\text{odd,even}} = [a_1, a_5]$ (indices 1, 5)
- $P_{\text{odd,odd}} = [a_3, a_7]$ (indices 3, 7)

Level 2: Base Cases and Combining ($n = 2$)
For $P_{\text{even,even}} = [a_0, a_4]$ with $\omega_2 = e^{-2\pi i/2} = -1$:

- Base case: $\text{FFT}([a_0], 1) = [a_0]$, $\text{FFT}([a_4], 1) = [a_4]$
- Combine:

$$\begin{aligned} y_0 &= a_0 + \omega_2^0 \cdot a_4 = a_0 + a_4 \\ y_1 &= a_0 - \omega_2^0 \cdot a_4 = a_0 - a_4 \end{aligned}$$

- Result: $[a_0 + a_4, a_0 - a_4]$

Similarly for $P_{\text{even,odd}} = [a_2, a_6]$:

- Result: $[a_2 + a_6, a_2 - a_6]$

Combine for P_{even} ($n = 4$) with $\omega_4^0 = 1$, $\omega_4^1 = -i$:

$$\begin{aligned} y_0 &= (a_0 + a_4) + \omega_4^0 \cdot (a_2 + a_6) = a_0 + a_4 + a_2 + a_6 \\ y_1 &= (a_0 - a_4) + \omega_4^1 \cdot (a_2 - a_6) = a_0 - a_4 - i(a_2 - a_6) \\ y_2 &= (a_0 + a_4) - \omega_4^0 \cdot (a_2 + a_6) = a_0 + a_4 - a_2 - a_6 \\ y_3 &= (a_0 - a_4) - \omega_4^1 \cdot (a_2 - a_6) = a_0 - a_4 + i(a_2 - a_6) \end{aligned}$$

Similarly compute for P_{odd}:

$$\begin{aligned} y_4 &= a_1 + a_5 + a_3 + a_7 \\ y_5 &= a_1 - a_5 - i(a_3 - a_7) \\ y_6 &= a_1 + a_5 - a_3 - a_7 \\ y_7 &= a_1 - a_5 + i(a_3 - a_7) \end{aligned}$$

Level 3: Final Combine ($n = 8$)

Combine results with $\omega_8^k = e^{-2\pi ik/8}$:

$$
\begin{aligned}
Y_0 &= y_0 + \omega_8^0 \cdot y_4 \\
Y_1 &= y_1 + \omega_8^1 \cdot y_5 \\
Y_2 &= y_2 + \omega_8^2 \cdot y_6 \\
Y_3 &= y_3 + \omega_8^3 \cdot y_7 \\
Y_4 &= y_0 - \omega_8^0 \cdot y_4 \\
Y_5 &= y_1 - \omega_8^1 \cdot y_5 \\
Y_6 &= y_2 - \omega_8^2 \cdot y_6 \\
Y_7 &= y_3 - \omega_8^3 \cdot y_7
\end{aligned}
$$

1.3.3 Recursive FFT Pseudocode

```
RecursiveFFT(a, n, omega_n):
    // a: coefficient array
    // n: length (must be power of 2)
    // omega_n: primitive n-th root of unity

    if n == 1:
        return [a[0]]

    // Split into even and odd indices
    even = [a[0], a[2], a[4], ..., a[n-2]]
    odd  = [a[1], a[3], a[5], ..., a[n-1]]

    // Recursive calls
    omega_half = omega_n * omega_n  // primitive (n/2)-th root
    y_even = RecursiveFFT(even, n/2, omega_half)
    y_odd  = RecursiveFFT(odd,  n/2, omega_half)

    // Combine using butterfly operations
    y = array of size n
    omega = 1  // omega_n^0
    for k = 0 to n/2 - 1:
        y[k]       = y_even[k] + omega * y_odd[k]
        y[k + n/2] = y_even[k] - omega * y_odd[k]
        omega = omega * omega_n  // next twiddle factor

    return y
```

1.3.4 Complexity Analysis

- **Time Complexity**: The recurrence $T(n) = 2T(n/2) + O(n)$ yields $T(n) = O(n \log n)$.
- **Space Complexity**: $O(n \log n)$ due to recursive call stack and temporary arrays allocated at each recursion level.

While conceptually elegant, the recursive implementation incurs significant overhead from function call management and stack frame allocation. Furthermore, the non-contiguous memory

access pattern reduces cache efficiency. These limitations motivate the iterative, in-place Cooley-Tukey algorithm presented in Section 2.

2 In-Place FFT Algorithm (Iterative Implementation)

We now present the iterative Cooley-Tukey algorithm, which eliminates recursion overhead and achieves $O(1)$ auxiliary space. Consider a polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ with $n = 2^k$ coefficients.

2.1 Bit-Reversal Permutation

The iterative algorithm begins by reordering the coefficients according to the bit-reversal permutation. For an index i with binary representation $(i_{k-1}i_{k-2}\cdots i_1 i_0)_2$, its bit-reversal $\mathrm{rev}(i)$ is $(i_0 i_1 \cdots i_{k-2} i_{k-1})_2$. The array element a_i is swapped with $a_{\mathrm{rev}(i)}$ for all $i < \mathrm{rev}(i)$ to avoid redundant operations.

For example, with $n = 8$ (3 bits):

Original Index & Binary & Bit-Reversed Index
0 & 000 & 0
1 & 001 & 4
2 & 010 & 2
3 & 011 & 6
4 & 100 & 1
5 & 101 & 5
6 & 110 & 3
7 & 111 & 7

The permuted array is:

$$[a_0, a_4, a_2, a_6, a_1, a_5, a_3, a_7]$$

2.2 Iterative Butterfly Operations

Starting with $m = 2$, double m each iteration until $m = n$. For each stage, we use twiddle factors:

$$\omega_m^j = e^{-2\pi i j/m} = \cos\left(\frac{2\pi j}{m}\right) - i \sin\left(\frac{2\pi j}{m}\right)$$

2.2.1 Stage 1: $m = 2$ (groups of size 2)

Process adjacent pairs with butterfly operations:

$$\begin{aligned}&[a_0 + a_4,\ a_0 - a_4,\ a_2 + a_6,\ a_2 - a_6,\ a_1 + a_5,\ a_1 - a_5,\ a_3 + a_7,\ a_3 - a_7]\\ &= [A_0^{(1)}, A_4^{(1)}, A_2^{(1)}, A_6^{(1)}, A_1^{(1)}, A_5^{(1)}, A_3^{(1)}, A_7^{(1)}]\end{aligned}$$

Note: $\omega_2^0 = 1$, so this stage uses only additions and subtractions.

2.2.2 Stage 2: $m = 4$ (groups of size 4)

Twiddle factors: $\omega_4^0 = 1$, $\omega_4^1 = -i$

For the first half $[A_0^{(1)}, A_4^{(1)}, A_2^{(1)}, A_6^{(1)}]$:

$$\begin{aligned}
A_0^{(2)} &= A_0^{(1)} + \omega_4^0 \cdot A_2^{(1)} \\
A_2^{(2)} &= A_0^{(1)} - \omega_4^0 \cdot A_2^{(1)} \\
A_4^{(2)} &= A_4^{(1)} + \omega_4^1 \cdot A_6^{(1)} \\
A_6^{(2)} &= A_4^{(1)} - \omega_4^1 \cdot A_6^{(1)}
\end{aligned}$$

For the second half $[A_1^{(1)}, A_5^{(1)}, A_3^{(1)}, A_7^{(1)}]$:

$$\begin{aligned}
A_1^{(2)} &= A_1^{(1)} + \omega_4^0 \cdot A_3^{(1)} \\
A_3^{(2)} &= A_1^{(1)} - \omega_4^0 \cdot A_3^{(1)} \\
A_5^{(2)} &= A_5^{(1)} + \omega_4^1 \cdot A_7^{(1)} \\
A_7^{(2)} &= A_5^{(1)} - \omega_4^1 \cdot A_7^{(1)}
\end{aligned}$$

2.2.3 Stage 3: $m = 8$ (groups of size 8, final stage)

Twiddle factors: $\omega_8^0 = 1$, $\omega_8^1 = \frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2}$, $\omega_8^2 = -i$, $\omega_8^3 = -\frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2}$

For all 8 elements:

$$\begin{aligned}
A_0^{(3)} &= A_0^{(2)} + \omega_8^0 \cdot A_1^{(2)} \\
A_1^{(3)} &= A_0^{(2)} - \omega_8^0 \cdot A_1^{(2)} \\
A_2^{(3)} &= A_2^{(2)} + \omega_8^1 \cdot A_3^{(2)} \\
A_3^{(3)} &= A_2^{(2)} - \omega_8^1 \cdot A_3^{(2)} \\
A_4^{(3)} &= A_4^{(2)} + \omega_8^2 \cdot A_5^{(2)} \\
A_5^{(3)} &= A_4^{(2)} - \omega_8^2 \cdot A_5^{(2)} \\
A_6^{(3)} &= A_6^{(2)} + \omega_8^3 \cdot A_7^{(2)} \\
A_7^{(3)} &= A_6^{(2)} - \omega_8^3 \cdot A_7^{(2)}
\end{aligned}$$

The final result $[A_0^{(3)}, A_1^{(3)}, A_2^{(3)}, A_3^{(3)}, A_4^{(3)}, A_5^{(3)}, A_6^{(3)}, A_7^{(3)}]$ is the DFT.

2.3 General In-Place FFT Pseudocode

```
FFT(a, n):
    // Step 1: Bit-reversal permutation
    for i = 0 to n-1:
        j = bit_reverse(i, log2(n))
        if i < j:
            swap(a[i], a[j])

    // Step 2: Iterative butterfly operations
    for m = 2; m <= n; m *= 2:        // m = current group size
        omega_m = exp(-2*pi*i / m)    // principal twiddle factor
        for k = 0; k < n; k += m:     // iterate over groups
```

```
omega = 1                    // omega_m^0
for j = 0; j < m/2; j++:  // butterfly pairs within group
    u = a[k + j]             // upper butterfly branch
    v = omega * a[k + j + m/2]  // lower butterfly branch
    a[k + j] = u + v         // in-place update
    a[k + j + m/2] = u - v
    omega = omega * omega_m    // update to next twiddle factor
```

And the inverse FFT can be implemented similarly, with the main difference being the use of the inverse twiddle factors and a final scaling step to divide by n.

2.4 Inverse FFT (IFFT)

The inverse FFT transforms the frequency-domain representation back to the time domain. The mathematical definition of the inverse DFT is:

$$a_k = \frac{1}{n} \sum_{j=0}^{n-1} A_j \cdot \omega_n^{-jk}$$

where $\omega_n^{-1} = e^{2\pi i/n}$ is the inverse primitive root of unity.

2.4.1 Key Differences from FFT

1. **Inverse twiddle factors**: Use $\omega_m^{-1} = e^{2\pi i/m}$ instead of $\omega_m = e^{-2\pi i/m}$
2. **Final scaling**: Divide all results by n after the butterfly operations

2.4.2 Step-by-Step Example: IFFT for $n = 8$

Given the DFT result $[A_0, A_1, A_2, A_3, A_4, A_5, A_6, A_7]$, we first apply bit-reversal permutation:

$$[A_0, A_4, A_2, A_6, A_1, A_5, A_3, A_7]$$

Stage 1: $m = 2$ (groups of size 2)
With inverse twiddle factor $\omega_2^0 = 1$:

$$\begin{aligned}&[A_0 + A_4,\ A_0 - A_4,\ A_2 + A_6,\ A_2 - A_6,\ A_1 + A_5,\ A_1 - A_5,\ A_3 + A_7,\ A_3 - A_7]\\ &= [B_0^{(1)}, B_4^{(1)}, B_2^{(1)}, B_6^{(1)}, B_1^{(1)}, B_5^{(1)}, B_3^{(1)}, B_7^{(1)}]\end{aligned}$$

Stage 2: $m = 4$ (groups of size 4)
Inverse twiddle factors: $\omega_4^0 = 1$, $\omega_4^{-1} = i$
For the first half:

$$\begin{aligned}B_0^{(2)} &= B_0^{(1)} + \omega_4^0 \cdot B_2^{(1)}\\ B_2^{(2)} &= B_0^{(1)} - \omega_4^0 \cdot B_2^{(1)}\\ B_4^{(2)} &= B_4^{(1)} + \omega_4^{-1} \cdot B_6^{(1)}\\ B_6^{(2)} &= B_4^{(1)} - \omega_4^{-1} \cdot B_6^{(1)}\end{aligned}$$

For the second half:

$$
\begin{aligned}
B_1^{(2)} &= B_1^{(1)} + \omega_4^0 \cdot B_3^{(1)} \\
B_3^{(2)} &= B_1^{(1)} - \omega_4^0 \cdot B_3^{(1)} \\
B_5^{(2)} &= B_5^{(1)} + \omega_4^{-1} \cdot B_7^{(1)} \\
B_7^{(2)} &= B_5^{(1)} - \omega_4^{-1} \cdot B_7^{(1)}
\end{aligned}
$$

Stage 3: $m = 8$ **(groups of size 8, final stage)**
Inverse twiddle factors: $\omega_8^0 = 1$, $\omega_8^{-1} = \frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2}$, $\omega_8^{-2} = i$, $\omega_8^{-3} = -\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2}$

$$
\begin{aligned}
B_0^{(3)} &= B_0^{(2)} + \omega_8^0 \cdot B_1^{(2)} \\
B_1^{(3)} &= B_0^{(2)} - \omega_8^0 \cdot B_1^{(2)} \\
B_2^{(3)} &= B_2^{(2)} + \omega_8^{-1} \cdot B_3^{(2)} \\
B_3^{(3)} &= B_2^{(2)} - \omega_8^{-1} \cdot B_3^{(2)} \\
B_4^{(3)} &= B_4^{(2)} + \omega_8^{-2} \cdot B_5^{(2)} \\
B_5^{(3)} &= B_4^{(2)} - \omega_8^{-2} \cdot B_5^{(2)} \\
B_6^{(3)} &= B_6^{(2)} + \omega_8^{-3} \cdot B_7^{(2)} \\
B_7^{(3)} &= B_6^{(2)} - \omega_8^{-3} \cdot B_7^{(2)}
\end{aligned}
$$

Final Scaling: Divide all results by $n = 8$:

$$
a_k = \frac{B_k^{(3)}}{8} \quad \text{for } k = 0, 1, \ldots, 7
$$

2.4.3 General In-Place IFFT Pseudocode

```
IFFT(a, n):
    // Step 1: Bit-reversal permutation (same as FFT)
    for i = 0 to n-1:
        j = bit_reverse(i, log2(n))
        if i < j:
            swap(a[i], a[j])

    // Step 2: Iterative butterfly operations with INVERSE twiddle factors
    for m = 2; m <= n; m *= 2:        // m = current group size
        omega_m = exp(2*pi*i / m)     // INVERSE principal twiddle factor
        for k = 0; k < n; k += m:     // iterate over groups
            omega = 1                  // omega_m^0
            for j = 0; j < m/2; j++:  // butterfly pairs within group
                u = a[k + j]           // upper butterfly branch
                v = omega * a[k + j + m/2]  // lower butterfly branch
                a[k + j] = u + v       // in-place update
                a[k + j + m/2] = u - v
                omega = omega * omega_m    // update to next twiddle factor
```

```
// Step 3: Final scaling (divide by n)
for i = 0 to n-1:
    a[i] = a[i] / n
```

2.4.4 Relationship between FFT and IFFT

The IFFT can also be computed using the same FFT routine with a clever trick:

1. Take the complex conjugate of the input: $\overline{A_j}$
2. Apply the standard FFT
3. Take the complex conjugate of the result
4. Divide by n

This works because:

$$\text{IFFT}(A)_k = \frac{1}{n}\sum_{j=0}^{n-1} A_j \omega_n^{-jk} = \frac{1}{n}\overline{\sum_{j=0}^{n-1} \overline{A_j}\omega_n^{jk}} = \frac{1}{n}\overline{\text{FFT}(\overline{A})_k}$$

This implementation is more elegant and efficient, as it avoids the overhead of recursion and uses only $O(1)$ additional space for temporary variables during the butterfly operations. The overall time complexity remains $O(n \log n)$.

2.5 Number Theoretic Transform

The Number Theoretic Transform (NTT) is a variant of the FFT that operates in the finite field $\mathbb{Z}_p$ for a prime p, thereby eliminating floating-point rounding errors. The NTT employs primitive roots of unity in $\mathbb{Z}_p$ in place of complex roots.

2.5.1 Theoretical Foundation

By Fermat's Little Theorem, for a prime p and any integer $a \not\equiv 0 \pmod{p}$:

$$a^{p-1} \equiv 1 \pmod{p}$$

A primitive root g modulo p is an element of multiplicative order $p-1$, meaning $\{g^0, g^1, \ldots, g^{p-2}\} \equiv \{1, 2, \ldots, p-1\} \pmod{p}$. For primes of the form $p = k \cdot 2^m + 1$, the element $g^{(p-1)/2^m}$ has multiplicative order 2^m, serving as the primitive 2^m-th root of unity for the NTT.

2.5.2 Parameter Selection

Practical NTT implementations require careful parameter selection:

- The modulus p must exceed all possible convolution results to prevent overflow.
- The transform length n must satisfy $n \mid (p-1)$ to ensure the existence of primitive n-th roots.

A widely used configuration is $p = 998244353 = 119 \cdot 2^{23} + 1$ with primitive root $g = 3$. This supports transforms up to length 2^{23} and enables efficient modular arithmetic using 32-bit integers. The NTT butterfly operation becomes:

$$\hat{a}_k = \left(a_k^{\text{even}} + \omega \cdot a_k^{\text{odd}}\right) \bmod p$$
$$\hat{a}_{k+n/2} = \left(a_k^{\text{even}} - \omega \cdot a_k^{\text{odd}}\right) \bmod p$$

where ω is the appropriate power of the primitive root.

3 Applications of the FFT

We now survey several applications of the FFT across diverse domains, illustrating the algorithm's versatility and fundamental importance in computational science.

3.1 Shor's Quantum Factoring Algorithm

Shor's algorithm is a quantum polynomial-time algorithm for integer factorization, providing an exponential speedup over classical methods. The algorithm relies critically on the Quantum Fourier Transform (QFT), a quantum analogue of the DFT implementable via quantum circuits.

3.1.1 Algorithm Overview

Given a composite integer N to be factored, the algorithm proceeds as follows:

1. Select a random integer $a \in \{2, \ldots, N-1\}$ with $\gcd(a, N) = 1$.
2. Define the modular exponentiation function $f(x) = a^x \bmod N$, which is periodic with some period r (the order of a modulo N).
3. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then

   $$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N)$$

   yield non-trivial factors of N.

3.1.2 Quantum Period Finding

The period r is found efficiently through quantum period finding:

1. Prepare a superposition over all $x \in \{0, \ldots, 2^n - 1\}$ where $2^n \geq N^2$.
2. Apply modular exponentiation to compute $f(x)$ in a second register.
3. Measure the second register, collapsing the first register to a superposition of states $\{|x_0 + kr\rangle : k \in \mathbb{Z}\}$ for some random x_0.
4. Apply the QFT to the first register, transforming the periodic superposition into a state where measurement yields $u \approx j \cdot 2^n/r$ with high probability.
5. Use continued fractions on $u/2^n$ to recover r.

The QFT for n qubits is defined as:

$$\text{QFT} : |x\rangle \mapsto \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i x y/2^n} |y\rangle$$

which structurally parallels the classical DFT. The QFT can be implemented using $O(n^2)$ quantum gates (or $O(n \log n)$ with approximate circuits), while classical FFT requires $O(n2^n)$ operations for equivalent precision, demonstrating quantum advantage.

The success probability of period finding is bounded below by a constant, ensuring that repeating the algorithm $O(1)$ times yields a factor with high probability.

3.2 Fast Convolution for FIR Filtering

Finite Impulse Response (FIR) filtering is a fundamental operation in digital signal processing where an input signal $x[n]$ of length N is convolved with a filter's impulse response $h[n]$ of length M to produce output $y[n]$:

$$y[n] = (x * h)[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n-k], \quad 0 \leq n \leq N + M - 2$$

Direct computation requires $O(NM)$ operations. For long signals, we employ fast convolution via the FFT to achieve $O((N + M) \log(N + M))$ complexity.

3.2.1 The Overlap-Add Method

When filtering streaming or very long signals, we partition $x[n]$ into blocks $x_m[n]$ of length L and apply circular convolution to each block:

1. Zero-pad the impulse response $h[n]$ to length $L + M - 1$ and compute $H[k] = \text{FFT}(h)$.
2. For each input block $x_m[n]$, compute $X_m[k] = \text{FFT}(x_m)$.
3. Multiply in frequency domain: $Y_m[k] = H[k] \cdot X_m[k]$.
4. Compute $y_m[n] = \text{IFFT}(Y_m)$.
5. Overlap and add: the last $M - 1$ samples of y_m overlap with the first $M - 1$ samples of y_{m+1}.

This method is equivalent to the Z-domain representation $Y(z) = H(z) \cdot X(z)$, where $H(z) = \sum_{k=0}^{M-1} h[k] z^{-k}$ is the transfer function. The FFT-based approach becomes advantageous when $L + M - 1 \gg M^2$.

3.3 Image Compression

Transform-based image compression techniques, notably the JPEG standard, exploit the frequency-domain representation of images to achieve efficient storage and transmission. While the JPEG standard employs the Discrete Cosine Transform (DCT) rather than the FFT, the underlying mathematical framework and computational principles are intimately connected to the FFT paradigm. We present a rigorous treatment of the transform-based compression pipeline.

3.3.1 Mathematical Framework

Given a grayscale image partitioned into non-overlapping 8×8 blocks, let $P[i][j]$ denote the pixel intensity at position (i, j) within a block, where $i, j \in \{0, 1, \ldots, 7\}$. The two-dimensional forward DCT is defined as:

$$F[u][v] = \frac{1}{4} C(u)C(v) \sum_{i=0}^{7} \sum_{j=0}^{7} P[i][j] \cos\left(\frac{(2i+1)u\pi}{16}\right) \cos\left(\frac{(2j+1)v\pi}{16}\right)$$

where $C(k) = 1/\sqrt{2}$ for $k = 0$ and $C(k) = 1$ otherwise. The coefficient $F[0][0]$ is termed the *DC coefficient*, representing the average intensity of the block, while the remaining 63 coefficients are *AC coefficients* encoding spatial frequency information.

3.3.2 Energy Compaction and DCT Advantages

The DCT is preferred over the DFT (and hence FFT) in lossy image compression for two fundamental reasons:

1. **Real-valued coefficients:** Unlike the DFT, which produces complex-valued coefficients requiring separate storage for real and imaginary components, the DCT yields purely real coefficients, reducing storage requirements by a factor of two.

2. **Superior energy compaction:** For natural images, which exhibit high spatial correlation, the DCT concentrates the signal energy into fewer low-frequency coefficients compared to the DFT. This property, known as *energy compaction*, enables more aggressive quantization of high-frequency components with minimal perceptual degradation.

3.3.3 Quantization and Lossy Compression

The compression achieved through transform coding is fundamentally lossy. Following the DCT, each coefficient $F[u][v]$ undergoes uniform quantization:

$$\hat{F}[u][v] = \text{round}\left(\frac{F[u][v]}{Q[u][v]}\right)$$

where $Q[u][v]$ denotes the entry in the *quantization matrix*. The JPEG standard defines default quantization matrices based on psychovisual studies, assigning larger step sizes to high-frequency components that are less perceptually significant. The quantization matrix can be scaled by a quality factor $q \in [1, 100]$ to control the bitrate-distortion trade-off.

3.3.4 Entropy Coding

The quantized coefficients undergo two-stage entropy coding:

1. **Zigzag scanning:** Coefficients are rearranged from the 8×8 matrix into a 64-element vector using a zigzag pattern that groups low-frequency coefficients before high-frequency ones, facilitating run-length encoding of the predominantly zero high-frequency values.

2. **Huffman coding:** The DC coefficients are differentially encoded (as differences between adjacent blocks), while AC coefficients are encoded using run-length encoding of zero runs followed by Huffman coding of the resulting symbols.

The inverse process at the decoder applies dequantization $\tilde{F}[u][v] = \hat{F}[u][v] \cdot Q[u][v]$ followed by the inverse DCT to reconstruct the pixel block. Note that due to quantization, $\tilde{F} \neq F$, and perfect reconstruction is impossible—this is the source of compression artifacts at low bitrates.

3.4 Spectral Convolution Networks

Recent advances in deep learning have leveraged the FFT to design efficient neural network layers that capture long-range dependencies. Zhang et al. [1] proposed Fast Fourier Convolution (FFC), a dual-stream architecture that combines traditional spatial convolutions with spectral operations.

3.4.1 Architecture Overview

The input tensor $X \in \mathbb{R}^{H \times W \times C}$ is split along the channel dimension into:

- **Local stream:** $X^l \in \mathbb{R}^{H \times W \times (1-\alpha_{in})C}$ processed by standard convolutions
- **Global stream:** $X^g \in \mathbb{R}^{H \times W \times \alpha_{in} C}$ processed by spectral convolutions

The outputs are computed through cross-domain interactions:

$$Y^l = f_l(X^l) + f_{g \to l}(X^g)$$
$$Y^g = f_g(X^g) + f_{l \to g}(X^l)$$

where f_l is a standard convolution, f_g is a spectral convolution, and the cross-terms $f_{g \to l}$, $f_{l \to g}$ are implemented via 1×1 convolutions.

3.4.2 Spectral Convolution Operation

The global stream applies the convolution theorem in the frequency domain:

$$\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g)$$

For real-valued signals, the FFT produces Hermitian-symmetric spectra, so the computation operates on both real and imaginary components:

$$z = \text{iFFT}\left(\text{ReLU}\left(\text{Conv}_{1\times1}\left(\text{Concat}(\text{FFT}(x)_{\text{real}}, \text{FFT}(x)_{\text{imag}})\right)\right)\right)$$

The spectral pathway enables receptive fields spanning the entire spatial domain with $O(HW \log(HW))$ complexity, compared to $O(HWk^2)$ for a $k \times k$ kernel that would require impractically large k to achieve equivalent coverage.

3.5 Orthogonal Frequency Division Multiplexing

Orthogonal Frequency Division Multiplexing (OFDM) is the dominant modulation scheme in modern broadband wireless systems, including IEEE 802.11 (Wi-Fi), 4G LTE, and 5G NR. OFDM partitions the available bandwidth into N orthogonal subcarriers, enabling spectrally efficient transmission and inherent resilience against frequency-selective fading.

3.5.1 Subcarrier Orthogonality

The subcarriers satisfy the orthogonality condition:

$$\frac{1}{N} \sum_{n=0}^{N-1} e^{2\pi i k n/N} e^{-2\pi i k' n/N} = \delta_{kk'}$$

where $\delta_{kk'}$ is the Kronecker delta. This orthogonality eliminates inter-carrier interference (ICI) without requiring guard bands between subcarriers.

3.5.2 Transmitter and Receiver Structure

At the transmitter, data symbols $X[k]$ modulate subcarriers via the IFFT:

$$x[n] = \text{IFFT}(X[k]) = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{2\pi ikn/N}, \quad 0 \leq n \leq N-1$$

The receiver applies the FFT to the received signal $y[n]$ (after synchronization and channel compensation):

$$Y[k] = \text{FFT}(y[n]) = \sum_{n=0}^{N-1} y[n] \cdot e^{-2\pi ikn/N}$$

The use of FFT/IFFT transforms the frequency-selective channel into N parallel flat-fading subchannels, enabling single-tap equalization per subcarrier. The cyclic prefix (CP) appended to each OFDM symbol converts linear convolution with the channel impulse response into circular convolution, preserving subcarrier orthogonality in multipath environments.

4 Conclusion

This report has presented a comprehensive treatment of the Fast Fourier Transform and its variants. We began with the mathematical foundations of polynomial multiplication via convolution, establishing the $O(n \log n)$ complexity bound achieved through the divide-and-conquer paradigm. The recursive and iterative Cooley-Tukey algorithms were detailed, with emphasis on the bit-reversal permutation and butterfly operations that enable cache-efficient, in-place computation.

We extended our analysis to the Number Theoretic Transform, demonstrating how the FFT framework generalizes to finite fields, yielding exact arithmetic solutions for polynomial convolution. The NTT's immunity to floating-point errors makes it indispensable in competitive programming and cryptographic applications.

Our survey of applications—spanning quantum computing (Shor's algorithm), digital signal processing (FIR filtering), image compression (DCT-based coding), deep learning (spectral convolution), and wireless communications (OFDM)—illustrates the remarkable versatility of the FFT framework. In each domain, the reduction from quadratic to near-linear complexity has enabled previously infeasible computational tasks.

The FFT stands as a paradigmatic example of how algorithmic insight can transform practical computation. Its continued relevance across emerging fields suggests that the Cooley-Tukey algorithm, despite its six-decade history, remains fundamental to the advancement of computational science.

References

[1] Lu Chi, Borui Jiang, and Yadong Mu. Fast fourier convolution. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.